

ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería en Sistemas

INFORME TÉCNICO

Gestión de Configuración y Estándares de Calidad

Correlación técnica aplicada al ecosistema YO-SOY-CRUD

Estándares analizados: IEEE 828-2012 · ISO/IEC/IEEE 12207:2017 · CMMI · ISO 9001:2015

Campo	Detalle
Repositorio	YO-SOY-CRUD (github.com/JonathanGuaman2004/YO-SOY-CRUD)
Sistemas analizados	esports-crud, audiohub-crud, missions-crud, pets-crud, epn-event-manager
Asignatura	Gestión de Configuración y Estándares de Calidad
Enfoque normativo	IEEE 828-2012 (SCM) · ISO/IEC/IEEE 12207:2017 (ciclo de vida) · CMMI (madurez de procesos) · ISO 9001:2015 (SGC)
Fecha	Junio de 2026
Integrantes	Maldonado Shirley- Jarrin Scarleth- Guamán Jonathan- Sigcha Christian

1. Introducción y alcance

El presente informe consolida el análisis de mantenimiento de software aplicado al ecosistema YO-SOY-CRUD desde cuatro marcos normativos complementarios: IEEE 828-2012 (Gestión de Configuración de Software), ISO/IEC/IEEE 12207:2017 (procesos del ciclo de vida del software), CMMI (modelo de madurez de procesos) e ISO 9001:2015 (Sistema de Gestión de Calidad). El ecosistema está compuesto por varios subsistemas CRUD contruidos con Node.js, Express y SQLite (esports-crud, audiohub-crud, missions-crud, pets-crud), todos ellos integrados vía HTTP con un hub centralizado de trazabilidad denominado epn-event-manager, desarrollado en NestJS con TypeORM.

Cada uno de los cuatro informes técnicos originales abordó el mismo conjunto de acciones de mantenimiento desde la óptica de su estándar correspondiente. Este documento integra esas perspectivas en una matriz de correlación única, identifica los Configuration Items (CI) comunes, clasifica las acciones de mantenimiento según su tipología, y presenta un análisis de cumplimiento normativo cruzado junto con una propuesta de mejora unificada para el siguiente ciclo de mantenimiento.

1.1 Objetivo general

Construir una matriz de correlación integral que vincule las acciones técnicas de mantenimiento aplicadas al ecosistema YO-SOY-CRUD con los conceptos de Gestión de Configuración de Software (SCM) y los requisitos de IEEE 828-2012, ISO/IEC/IEEE 12207:2017, CMMI e ISO 9001:2015, evaluando el nivel de cumplimiento actual bajo cada estándar y proponiendo mejoras consolidadas para el siguiente ciclo.

1.2 Objetivos específicos

- Identificar y catalogar los Configuration Items (CI) presentes en los subsistemas analizados.
- Clasificar cada acción de mantenimiento según su tipo: correctivo, adaptativo, perfectivo o preventivo.
- Definir los conceptos SCM aplicados: CI, baseline, change request, status accounting y configuration audit.
- Vincular las acciones técnicas con las actividades SCM de IEEE 828-2012 (Identification, Control, Status Accounting, Auditing).
- Relacionar las acciones con los procesos del ciclo de vida de ISO/IEC/IEEE 12207:2017.
- Evaluar el nivel de madurez académico estimado bajo el modelo CMMI.
- Vincular las acciones con las cláusulas relevantes de ISO 9001:2015 (§6.1, §7.5, §8.1, §8.5.1, §8.5.6, §8.6, §9.1, §10.2).
- Evaluar el nivel de cumplimiento normativo global y proponer acciones preventivas consolidadas para el siguiente ciclo.

1.3 Alcance del análisis

El análisis abarca los subsistemas del repositorio YO-SOY-CRUD: esports-crud, audiohub-crud, missions-crud, pets-crud y epn-event-manager. Para los CRUDs REST se revisan principalmente server.js, las interfaces web (public/index.html), package.json, archivos .env/.env.example, las suites de pruebas (test/*.test.js) y los registros de mantenimiento (Mantenimientos.md). Para epn-event-manager se analizan src/main.ts, src/app.module.ts, src/database/database.module.ts, las entidades TypeORM, los DTOs de validación, events.service.ts, events.service.spec.ts, tsconfig.json y eslint.config.mjs.

2. Marco normativo: los cuatro estándares

Cada uno de los cuatro estándares aporta una perspectiva complementaria sobre el mismo conjunto de acciones técnicas de mantenimiento. La siguiente tabla resume el enfoque de cada uno y su relación con la Gestión de Configuración de Software (SCM).

Estándar	Enfoque principal	Relación con SCM
IEEE 828-2012	Define los requisitos mínimos para un Plan de Gestión de Configuración de Software (SCM Plan): identificación de CIs, control de cambios, status accounting y auditoría de configuración.	Es el estándar SCM por excelencia; las otras tres normas se apoyan en sus cuatro actividades (Identification, Control, Status Accounting, Auditing).
ISO/IEC/IEEE 12207:2017	Marco integral de procesos del ciclo de vida del software (requisitos, diseño, implementación, verificación, validación, operación, mantenimiento, gestión de configuración).	Incluye la Gestión de Configuración (cláusula 6.3.5) como uno de los procesos de soporte del ciclo de vida completo.
CMMI	Modelo de madurez y capacidad de procesos organizado en áreas de práctica (Configuration Management, Verification & Validation, Risk Management, etc.).	Configuration Management (CM) es un área de práctica nuclear; su madurez determina en gran medida el nivel de capacidad organizacional alcanzable.
ISO 9001:2015	Sistema de Gestión de Calidad orientado a procesos, riesgos y mejora continua, con énfasis en control de producción, información documentada y no conformidades.	El control de cambios (§8.5.6) y la información documentada (§7.5) son, en esencia, requisitos de SCM expresados desde la óptica de calidad organizacional.

2.1 Las cuatro actividades SCM de IEEE 828-2012

Actividad SCM	Descripción
SCM Identification	Identificar, nombrar y versionar todos los elementos bajo control de configuración (Configuration Items, CI).
SCM Control	Gestionar formalmente los cambios sobre los CI: solicitud, evaluación, aprobación e implementación controlada.
SCM Status Accounting	Registrar y reportar el estado de cada CI a lo largo del ciclo de vida del proyecto.
SCM Auditing	Verificar que los CI cumplen sus requisitos funcionales y que los cambios aprobados se han implementado correctamente.

3. Contexto técnico del ecosistema mantenido

YO-SOY-CRUD es un ecosistema distribuido de servicios que se integran vía HTTP. Cada CRUD expone un API REST protegida por API Key (header X-FIS-EPN-KEY) y registra cada mutación enviando un evento al epn-event-manager, que centraliza la trazabilidad de operaciones de todos los CRUDs conectados, enrutando cada evento a una tabla SQLite diferente según su tipo de acción (CREATE, UPDATE, DELETE, QUERY).

3.1 Inventario de subsistemas

Subsistema	Tecnología	Función principal
esports-crud	Node.js 22 + Express 5 + SQLite nativo (node:sqlite)	CRUD de torneos de deportes electrónicos. Protección por API Key, logging JSON estructurado e integración con el hub de eventos.
audiohub-crud	Node.js + Express + SQLite nativo	CRUD de biblioteca de audios (canciones y podcasts). Integración con el Event Manager vía Axios.

Subsistema	Tecnología	Función principal
missions-crud	Node.js + Express + SQLite + dotenv + Axios	CRUD de misiones espaciales (Space Mission Control). Persistencia con WAL, índices y envío de eventos al hub.
pets-crud	Node.js + Express + SQLite nativo	CRUD de gestión de mascotas con seguridad por API Key, logger personalizado y pruebas de API.
epn-event-manager	NestJS 11 + TypeORM + better-sqlite3	Hub central de eventos. Recibe y almacena trazas de operaciones (CREATE, UPDATE, DELETE, QUERY) de cualquier sistema externo conectado.

3.2 Configuration Items (CI) identificados

A continuación se presenta el inventario consolidado de Configuration Items detectados en el ecosistema, agrupados por categoría.

CI	Tipo	Subsistema(s)
server.js	Código fuente principal (API Express, validaciones, seguridad, CRUD)	esports-crud / audiohub-crud / missions-crud / pets-crud
public/index.html (+ app.js)	Interfaz web, fetch con headers de seguridad, escapeHtml	Todos los CRUDs
.env / .env.example	Configuración de entorno (PORT, DB_PATH, API_KEY, EVENT_MANAGER_URL)	Todos los subsistemas
package.json / package-lock.json	Manifiesto de dependencias y versiones congeladas	Todos los subsistemas
*.sqlite (db/)	Base de datos (esports.sqlite, audiohub.sqlite, missions.sqlite, events.sqlite)	Cada subsistema
Mantenimientos.md	Documentación SCM / registro formal de mantenimientos	Todos los subsistemas
test/*.test.js / *.spec.ts	Suites de pruebas unitarias y de API (node:test / Jest)	Todos los subsistemas
logs/*.log (audit.log, audiohub.log, esports-audit.log)	Log de auditoría JSON estructurado	Todos los subsistemas
app.module.ts / main.ts	Módulo raíz NestJS, bootstrap, ValidationPipe global, CORS	epn-event-manager
database.module.ts	Configuración TypeORM	epn-event-manager
*.entity.ts (x4)	Entidades ORM: create_events, update_events, delete_events, query_events	epn-event-manager
create-event.dto.ts	DTO de validación de entrada (class-validator)	epn-event-manager
events.service.ts / events.service.spec.ts	Servicio de negocio central y pruebas Jest	epn-event-manager
tsconfig.json / eslint.config.mjs	Configuración TypeScript y reglas de estilo	epn-event-manager
.github/workflows/ci.yml	Pipeline de calidad (Configuration Audit automatizado)	Ecosistema

4. Conceptos SCM aplicados

Los seis conceptos fundamentales de SCM, según IEEE 828-2012, se aplican de forma transversal en los cuatro informes originales. La siguiente tabla resume su definición y su aplicación concreta en el ecosistema YO-SOY-CRUD.

Concepto SCM	Definición aplicada	Aplicación en YO-SOY-CRUD
Configuration Item (CI)	Elemento de software identificado y controlado formalmente dentro del SCM.	Los CI del inventario (sección 3.2): archivos de código, configuración, pruebas, bases de datos, entidades ORM y documentación.
Baseline (línea base)	Versión estable tomada como referencia formal para el control de cambios.	Estado del repositorio main después de aplicar los cuatro tipos de mantenimiento documentados en Mantenimientos.md, con pruebas verdes y evidencia de ejecución.
Change Request	Solicitud formal que justifica una intervención sobre la configuración existente.	Cada bloque de Mantenimientos.md: incorporar API Key, reforzar validaciones, implementar logger, agregar pruebas, externalizar variables de entorno, integrar con el Event Manager.
Change Item	Cambio puntual dentro de una solicitud de cambio más amplia.	Corrección de clean() para objetos; modificación de fetch() para incluir X-FIS-EPN-KEY; adición de @IsIn en el DTO de eventos; aplicación de escapeHtml() en el frontend.
Status Accounting	Registro del estado y trazabilidad de cada CI a lo largo del tiempo.	Logs JSON (audit.log, esports-audit.log, audiohub.log) con timestamp ISO 8601; 4 tablas de eventos en epn-event-manager; endpoints /stats y /health.
Configuration Audit	Revisión técnica para confirmar que la nueva baseline cumple sus requisitos.	Ejecución de npm run check, npm run lint, npm test y npm run build antes de aceptar la baseline; ejecución de events.service.spec.ts con mocks de repositorio.

4.1 Los cuatro tipos de mantenimiento

La taxonomía de mantenimiento aplicada de forma consistente en los cuatro informes clasifica cada acción técnica según su naturaleza:

- **Correctivo:** corrige defectos detectados en producción o pruebas (p. ej., el bug de clean() que retornaba "[object Object]" para campos tipo objeto).
- **Adaptativo:** adapta el sistema a un nuevo entorno, política o requisito institucional (p. ej., incorporación de API Key con header X-FIS-EPN-KEY, externalización de variables de entorno con dotenv).
- **Perfectivo:** mejora el sistema sin corregir errores existentes (p. ej., logger estructurado JSON con niveles INFO/WARN/ERROR, normalizeEvents() para unificar resultados de múltiples tablas).
- **Preventivo:** anticipa problemas futuros mediante salvaguardas (p. ej., suite de 37 pruebas unitarias en esports-crud, sanitización de entrada, escapeHtml() contra XSS, validación de DTOs con whitelist).

5. Matriz de correlación técnica

La siguiente matriz integra las acciones técnicas más representativas observadas a lo largo del ecosistema, correlacionándolas simultáneamente con: el tipo de mantenimiento, el concepto SCM asociado, la actividad de IEEE 828-2012, el proceso de ISO/IEC/IEEE 12207:2017, el área de práctica CMMI y la cláusula de ISO 9001:2015 que respalda o exige dicha acción. Esta es la pieza central del análisis, ya que demuestra que una misma acción de código satisface simultáneamente requisitos de los cuatro marcos normativos.

ID	Acción técnica	Tipo mant.	Concepto SCM	IEEE 828	ISO 12207	CMMI	ISO 9001
A1	Corrección de clean(): guarda de tipo para evitar "[object Object]" en campos de objeto.	Correctivo	Change Item / CI Quality	SCM Control	\$6.4.13 Mantenimiento	VV — Verification & Validation	\$10.2 No conformidad y acción correctiva
A2	Logger estructurado (INFO/WARN/ERROR) con	Correctivo / Perfectivo	Status Accounting /	SCM Status Accounting	\$6.3.6 Gestión de Información	MPM — Meas.	\$7.5 Información documentada

ID	Acción técnica	Tipo mant.	Concepto SCM	IEEE 828	ISO 12207	CMMI	ISO 9001
	timestamp ISO 8601 a consola y archivo de auditoría.		Information Item			Performance & Measurement	
A3	Middleware requireApiKey (header X-FIS-EPN-KEY) con rechazo 401 y registro WARN en endpoints CRUD.	Adaptativo / Preventivo	Security CI / Change Request	SCM Control	§6.4.5 / §6.4.7 Diseño e Implementación	RDM, TS, RSK	§8.5.1 Control de producción
A4	Externalización de variables de entorno (PORT, DB_PATH, API_KEY, EVENT_MANAGER_URL) vía dotenv y .env.example.	Adaptativo	Environment Baseline / Configuration Item	SCM Identification	§6.2.2 / §6.3.5 Gestión de infraestructura y configuración	CM — Configuration Management	§8.1 Planificación operacional
A5	Validación de orden de fechas (date_end > date_start) como nueva regla de negocio.	Adaptativo	Change Request / Validation CI	SCM Control	§6.4.9 Verificación	VV, PQA	§8.6 Liberación de productos
A6	Suite de pruebas unitarias (37 en esports-crud, suites equivalentes en otros CRUDs) cubriendo validaciones y normalizaciones.	Perfectivo	Verification Baseline / Configuration Audit	SCM Auditing	§6.4.9 Verificación	VV — Verification & Validation	§8.6 Liberación de productos
A7	Sanitización preventiva: rechazo de <>{}, límites de longitud y rangos numéricos, try-catch granular por ruta.	Preventivo	Risk Control / Quality CI	SCM Control + SCM Auditing	§6.3.4 Gestión de riesgos	RSK, PQA	§6.1 Riesgos y oportunidades
A8	CreateEventDto con @IsIn, @MaxLength, @NotEmpty; ValidationPipe global con whitelist + forbidNonWhitelisted.	Preventivo / Correctivo	CI Quality / Change Item	SCM Control	§6.4.9 Verificación / §6.4.11 Validación	VV, PQA, RDM	§8.5.1 Control de producción
A9	Diseño de 4 tablas separadas por acción en epn-event-manager (CREATE/UPDATE/DELETE/QUERY) — deuda técnica intencional.	Diseño inicial con deuda técnica	Product Baseline — CI deficiente	SCM Identification	§6.3.5 Gestión de configuración	CM — Configuration Management	§10.2 No conformidad y acción correctiva
A10	normalizeEvents(): unifica resultados de las 4 tablas y ordena por fecha descendente con metadatos de trazabilidad.	Perfectivo	Baseline Update / Status Accounting	SCM Status Accounting	§6.4.12 Operación	TS, PI, MPM	§7.5 Información documentada
A11	originPatternToRegExp(): parseo seguro de rangos de puertos CORS configurable por variable de entorno CORS_ORIGINS.	Adaptativo	Environment Control / Change Request	SCM Control	§6.4.10 Transición	Environment Control	§8.5.6 Control de cambios
A12	Suite de pruebas Jest con mocks de repositorio: enrutamiento por acción, validación de payload, estadísticas.	Perfectivo	Configuration Audit / Verification Baseline	SCM Auditing	§6.4.9 Verificación	VV, PQA	§8.6 Liberación de productos
A13	escapeHtml() aplicado antes de renderizar datos de usuario en las tarjetas del frontend (mitigación XSS).	Preventivo	Secure CI / Risk Management	SCM Control	§6.4.5 / §6.4.7 Diseño e Implementación	TS, ESEC, RSK	§8.5.1 Control de producción
A14	migrateDatabase(): PRAGMA WAL + foreign_keys=ON + índices automáticos ejecutados de forma idempotente al arranque.	Preventivo	Risk Mitigation Infraestructura	SCM Plan — Reproducibilidad del entorno	§6.4.10 Transición	CM, RSK	§8.1 Planificación operacional
A15	sendEvent() vía Axios hacia epn-event-manager en cada operación CRUD, con manejo de timeout y errores.	Perfectivo / Correctivo	Change Item / Interface CI	SCM Status Accounting	§6.4.8 Integración	PI, MC, MPM	§9.1 Seguimiento y medición
A16	Scripts npm check / lint / test / build y workflow de GitHub Actions como pipeline de calidad.	Perfectivo	Configuration Audit / Baseline Approval	SCM Auditing	§6.3.5 / §6.3.8 Gestión de configuración y QA	CM, PQA, II	§8.6 Liberación de productos

6. Análisis de cumplimiento por estándar

El análisis considera tres niveles: Cumple, cuando existe evidencia técnica directa en el repositorio; Parcial, cuando existe implementación pero falta formalización, documentación completa o automatización; y No cumple / Pendiente, cuando no se encontró evidencia suficiente.

6.1 IEEE 828-2012 — Gestión de Configuración de Software

Actividad IEEE 828-2012	Nivel	Evidencia / Mejora requerida
SCM Identification — Catálogo de CI	Cumple	CI's identificados y nombrados; IDs únicos generados programáticamente (p. ej. TRN-XXXX, MSN-XXXX); .env.example documenta CI's de configuración. Mejora: agregar git tags y CHANGELOG.md.
SCM Identification — Versionado de dependencias	Cumple	package.json + package-lock.json congelan versiones exactas en todos los subsistemas. Mejora: automatizar revisión con Dependabot.
SCM Control — Gestión formal de cambios	Parcial	Cambios documentados en Mantenimientos.md con tipo, CI afectado e impacto; no existe flujo formal de Change Request con Issue, rama y PR. Mejora: PR templates, etiquetas y CHANGELOG.md.
SCM Control — Protección de CI críticos	Cumple	requireApiKey() impide modificaciones no autorizadas; ValidationPipe con whitelist rechaza entradas mal formadas; clean() previene corrupción de datos. Mejora: Branch Protection en GitHub.
SCM Status Accounting — Registro cronológico	Cumple	Logs JSON con timestamp ISO 8601 (INFO/WARN/ERROR); tablas de eventos con campos de fecha independientes. Mejora: rotación y retención de logs en producción.
SCM Status Accounting — Reporte de estado	Cumple	Endpoints /stats y /health retornan métricas en tiempo real y disponibilidad. Mejora: dashboard de métricas y alertas sobre niveles WARN/ERROR.
SCM Auditing — Pruebas como auditoría funcional	Cumple	Suites de pruebas unitarias (37 en esports-crud) verifican los CI de validación; events.service.spec.ts verifica el CI de servicio. Mejora: automatizar en CI/CD ante cada PR.
SCM Auditing — Auditoría de acceso e integridad de entrada	Cumple	requireApiKey() registra intentos fallidos (WARN); ValidationPipe con whitelist + forbidNonWhitelisted garantiza integridad del DTO. Mejora: rate limiting y contract testing.
SCM Auditing — Deuda técnica documentada	Parcial	Las 4 tablas separadas de epn-event-manager están identificadas como inconsistencia intencional en comentarios; no existe registro formal de no conformidad. Mejora: Change Request con plan de refactorización a tabla única "events" con columna action.
SCM Plan — Documentación y reproducibilidad	Parcial	Mantenimientos.md actúa como SCM Plan informal; JSDoc y README cubren ejecución. Falta .env.example publicado en algunos CRUDs y runbook operacional.

6.2 ISO/IEC/IEEE 12207:2017 — Procesos del ciclo de vida

La norma ISO/IEC/IEEE 12207:2017 fue retirada y revisada por ISO/IEC/IEEE 12207:2026; para efectos académicos este informe usa la estructura de procesos de la versión 2017, según se acordó en clase.

Proceso ISO 12207	Nivel	Evidencia / Mejora requerida
6.3.4 Gestión de Riesgos	Parcial-alto	Mitigación vía API-Key, validaciones, escapeHtml y try-catch; falta registro formal de riesgos y matriz de amenazas.
6.3.5 Gestión de Configuración	Parcial	CI's identificados, .gitignore protege secretos; falta CHANGELOG.md, etiquetas de versión, flujo formal de Change Request y pipeline CI/CD completo.

Proceso ISO 12207	Nivel	Evidencia / Mejora requerida
6.3.6 Gestión de Información	Parcial	README y documentación de API existen pero son básicos; falta documentación completa de endpoints, errores y ejemplos de respuesta.
6.3.8 Aseguramiento de Calidad	Parcial-alto	Pruebas con node:test/Jest y scripts npm disponibles; falta ejecución automática en CI y reporte de cobertura.
6.4.7 Implementación	Cumple	Cambios implementados consistentemente en backend y frontend: middleware, logger, validaciones, pruebas y manejo de errores.
6.4.8 Integración	Parcial	Integración funcional entre CRUDs y epn-event-manager vía Axios/sendEvent(); falta prueba automatizada de integración end-to-end.
6.4.9 Verificación	Cumple parcial-alto	Pruebas unitarias y de API para seguridad, validación y flujo CRUD; scripts check/lint/test disponibles. Mejora: ejecutar en Pull Requests.
6.4.10 Transición	Parcial	README explica ejecución y puertos; falta .env.example completo, guía de despliegue y colección Postman en algunos subsistemas.
6.4.11 Validación	Parcial	Se validan reglas técnicas (longitudes, tipos, fechas); falta documentar criterios de aceptación funcionales con el usuario.
6.4.12 Operación	Parcial	Endpoints /health y manejo de Event Manager offline existen; falta monitoreo continuo, rotación de logs y guía operacional.
6.4.13 Mantenimiento	Cumple parcial-alto	Se evidencian acciones correctivas, adaptativas, perfectivas y preventivas documentadas formalmente en Mantenimientos.md.

6.3 CMMI — Áreas de práctica

La calificación de madurez es académica y no representa una evaluación oficial CMMI; una valoración formal requeriría una appraisal con un Lead Appraiser autorizado. CMMI se utiliza aquí como guía de mejora de procesos y trazabilidad.

Área CMMI	Nivel académico estimado	Justificación
CM — Configuration Management	Parcial-alto	Existen CIs identificados, pipeline CI/CD parcial y repositorio organizado; falta CHANGELOG, tags de versión, .env.example completo y PR template.
VV — Verification & Validation	Alto	Pruebas unitarias y funcionales con resultados aprobados en todos los subsistemas revisados (ej.: 4/4 en missions-crud, 8/8 en epn-event-manager, 37/37 en esports-crud).
PQA — Process Quality Assurance	Alto	Scripts de check/lint/test ejecutados correctamente; ValidationPipe y DTOs garantizan calidad de entrada.
MPM — Managing Performance and Measurement	Parcial-alto	Endpoints /stats, /events y logs de auditoría permiten medir actividad; falta dashboard histórico y umbrales de alerta.
RSK — Risk Management	Parcial	Mitigación de seguridad básica (API-Key, validaciones, escapeHtml); falta matriz formal de riesgos y análisis de amenazas documentado.
GOV / II — Governance & Implementation Infrastructure	Parcial	Existe workflow de GitHub Actions; falta política de ramas, Branch Protection y Definition of Done formal.

6.4 ISO 9001:2015 — Sistema de Gestión de Calidad

Cláusula ISO 9001:2015	Nivel	Evidencia / Mejora requerida
§6.1 Riesgos y oportunidades	Parcial	validatePayloadForAction(), requireApiKey y validaciones mitigan riesgos identificados; no existe registro formal de análisis de riesgos. Mejora: matriz de riesgos con probabilidad, impacto y plan de mitigación.
§7.5 Información documentada	Parcial-alto	Logger JSON a archivo, README con instrucciones, colecciones Postman exportadas. Mejora: completar API_DOCS, agregar .env.example y mantener changelog por ciclo.
§8.1 Planificación y control operacional	Cumple	migrateDatabase() ejecuta configuración de WAL, foreign keys e índices al arranque de forma automática y reproducible. Mejora: documentar el proceso en un runbook operacional.
§8.5.1 Control de producción	Cumple	requireApiKey() controla acceso; validateAudio()/validateTournament()/validateMission() y CreateEventDto verifican datos antes de persistir; ValidationPipe global en NestJS. Mejora: pruebas de contrato (contract testing).
§8.5.6 Control de cambios	Parcial	originPatternToRegExp() gestiona cambios de política CORS de forma controlada y configurable; no existe proceso formal de Change Request ni CHANGELOG.md. Mejora: implementar CHANGELOG.md, PR templates y Branch Protection.
§8.6 Liberación de productos	Cumple	Las funciones de validación rechazan entidades inválidas antes de INSERT; los DTOs rechazan payloads incorrectos antes de guardar. Mejora: scripts de npm run check en CI/CD para bloquear releases con pruebas fallidas.
§9.1 Seguimiento, medición, análisis y evaluación	Cumple	sendEvent() registra cada operación CRUD en epn-event-manager; endpoints /stats exponen métricas de uso; logger JSON para auditoría. Mejora: dashboard de métricas y alertas sobre niveles WARN/ERROR.
§10.2 No conformidad y acción correctiva	Parcial	Las 4 tablas separadas de epn-event-manager están identificadas como deuda técnica intencional documentada en comentarios de las entidades. Mejora: registrar formalmente la no conformidad, planificar refactorización a tabla única "events" con columna action y auditar el cambio.

7. Deudas técnicas identificadas y tratamiento

La deuda técnica no se limita a fallos funcionales: abarca también seguridad, configuración, trazabilidad, documentación, pruebas y decisiones de diseño que dificultan el mantenimiento futuro. La siguiente tabla consolida las deudas técnicas detectadas en el ecosistema y su estado de tratamiento.

Deuda técnica	Situación detectada	Tratamiento aplicado / recomendado	Estado
Bug en clean() con objetos JS	Retornaba "[object Object]" si el campo recibía un objeto, corrompiendo la base de datos.	Guarda de tipo: if (typeof value === "object") return "".	Mitigada
Logger sin estructura ni niveles	console.log sin timestamp, nivel ni formato JSON impedía la auditoría.	Logger estructurado con niveles INFO/WARN/ERROR, formato ISO 8601 y escritura a archivo.	Mitigada
Endpoints sin autenticación	Endpoints POST/PUT/DELETE accesibles sin control de acceso.	Middleware requireApiKey con rechazo 401 y registro WARN de cada intento fallido.	Mitigada
Credenciales hardcodeadas	API Key y URLs del Event Manager incluidas en el código fuente.	.env + .env.example; variables de entorno cargadas con dotenv / --env-file.	Mitigada

Deuda técnica	Situación detectada	Tratamiento aplicado / recomendado	Estado
Sin validación de fechas / payloads	Posibilidad de registros con fechas inconsistentes o payloads vacíos.	Validación <code>date_end > date_start</code> ; <code>validatePayloadForAction()</code> exige payload en CREATE/UPDATE/DELETE.	Mitigada
Sin pruebas unitarias	Ninguna cobertura de los Cls de validación.	Suites de pruebas unitarias y de API (node:test / Jest) cubriendo validaciones, seguridad y flujo CRUD completo.	Mitigada
Sin sanitización de entrada / riesgo XSS	Campos sin límites de longitud; interfaz renderizaba datos de usuario sin escapar HTML.	Restricciones de longitud, rechazo de <code><>{} </code> , rangos numéricos acotados; <code>escapeHtml()</code> antes de renderizar.	Mitigada
DTO sin validación estricta	CreateEventDto aceptaba acciones inválidas y campos extra.	<code>@IsIn</code> , <code>@IsNotEmpty</code> , <code>@MaxLength</code> ; <code>whitelist + forbidNonWhitelisted</code> en <code>ValidationPipe</code> global.	Mitigada
4 tablas separadas en epn-event-manager (deuda intencional)	Diseño con tablas independientes por acción genera duplicación de código y dificulta consultas transversales.	Documentado como inconsistencia intencional; <code>normalizeEvents()</code> mitiga el impacto en consultas, pero la refactorización sigue pendiente.	Pendiente de refactorización
Secreto local versionado (.env)	Se detectó .env incluido en el paquete de algunos CRUDs.	Eliminar .env del repositorio, mantener .env.example sin secretos reales y usar GitHub Secrets en CI/CD.	Pendiente
Sin control formal de cambios	Los cambios no pasan por un flujo de Issue → rama → PR → revisión → merge.	Documentados en Mantenimientos.md; se propone GitHub Issues, PR templates y CHANGELOG.md (ver sección 8).	Parcial
Documentación de API incompleta	README y API_DOCS existen pero son básicos; faltan cuerpos de error y ejemplos de respuesta.	Completar API_DOCS.md, agregar colección Postman completa y mantener documentación viva por cada ciclo.	Parcial

8. Acciones preventivas para el siguiente ciclo

Las siguientes acciones preventivas consolidan las propuestas de los cuatro informes originales y se priorizan por su impacto en evitar que la deuda técnica regrese, alineándolas simultáneamente con los cuatro estándares analizados.

8.1 Control formal de cambios con Change Request

Cada intervención debe iniciar como un Issue en GitHub que indique: tipo de mantenimiento, CI afectado, riesgo identificado, pruebas requeridas y evidencia esperada. Al aprobarse el cambio, se crea una rama, se realiza un Pull Request y se registra en CHANGELOG.md.

- Evita cambios no rastreados sobre Cls críticos y permite reconstruir la historia completa de la configuración.
- Herramientas: GitHub Issues, PR templates, CHANGELOG.md con Conventional Commits.
- Cobertura normativa: IEEE 828 SCM Control · ISO 12207 §6.3.5 · CMMI CM · ISO 9001 §8.5.6.

8.2 Pipeline CI/CD como auditoría automatizada de CM

Automatizar `npm install`, `npm run check`, `npm run lint` y `npm test` en cada Pull Request antes de unir a main. Para epn-event-manager: `tsc --noEmit + eslint + jest --coverage`. Activar Branch Protection para bloquear merges directos sin revisión aprobada.

- Garantiza que ningún CI degradado llegue a la baseline; formaliza la Configuration Audit de IEEE 828-2012.
- Herramientas: GitHub Actions, Branch Protection, Codecov/c8, Dependabot.
- Cobertura normativa: IEEE 828 SCM Auditing · ISO 12207 §6.4.9 · CMMI VV/PQA · ISO 9001 §8.6 / §9.1.

8.3 Documentación viva alineada a los cuatro estándares

Actualizar Mantenimientos.md, README.md, .env.example, API_DOCS.md y la matriz de correlación en cada ciclo como parte de la Definition of Done. Registrar la deuda técnica de las 4 tablas del Event Manager como Change Request formal con plan de refactorización a tabla única "events" con columna action.

- Evita que el código avance más rápido que la documentación del SCM Plan; facilita auditorías futuras.
- Herramientas: Markdown, carpeta docs/evidencias, colección Postman, definición de Done.
- Cobertura normativa: ISO 12207 §6.3.6 · CMMI CM/GOV · ISO 9001 §7.5 · IEEE 828 SCM Plan.

8.4 Definition of Done recomendada

- El ticket/Change Request tiene requisito, criterio de aceptación y CI afectado documentados.
- El código está en una rama separada y pasa por Pull Request con revisión de pares.
- Se ejecutan npm run check, npm run lint, npm test y npm run build (cuando aplique) con resultado exitoso.
- Se actualiza README, API_DOCS, .env.example y/o colección Postman si el cambio afecta endpoints o configuración.
- Se adjunta evidencia de ejecución: captura, log, resultado de pruebas o endpoint de monitoreo.
- Se actualiza CHANGELOG.md y se marca la nueva baseline con un git tag (p. ej. v1.0-maintenance).

9. Respuestas a preguntas de control

9.1 ¿Cómo se evita que la deuda técnica regrese?

La deuda técnica regresa cuando los equipos no tienen criterios claros de aceptación. Para prevenirlo se establece una Definition of Done para mantenimiento: ningún cambio entra a main sin Issue documentado, rama de trabajo, revisión de pares, pruebas actualizadas (npm test + npm run check), documentación actualizada en Mantenimientos.md y evidencia de integración con epn-event-manager. Los CIs críticos deben versionarse con git tags y auditarse antes de convertirse en nueva baseline. IEEE 828-2012 exige que la SCM Auditing confirme que los CIs cumplen sus requisitos antes de aprobar la nueva baseline, mientras que ISO 9001 §10.2 exige identificar no conformidades, analizar su causa raíz y planificar la acción correctiva de forma formal, no anecdótica.

9.2 ¿Qué herramientas de CI/CD automatizan la CM?

Las herramientas recomendadas para todo el ecosistema YO-SOY-CRUD son:

- Git con Branch Protection: bloquea merges directos a main sin revisión aprobada.
- GitHub Actions: ejecuta npm run check + npm test (y npm run build para epn-event-manager) en cada Pull Request.
- Codecov o c8: genera reportes de cobertura de pruebas y puede fallar el PR si cae por debajo del umbral.
- Dependabot: detecta dependencias desactualizadas o con vulnerabilidades conocidas.
- CHANGELOG.md con Conventional Commits: documenta automáticamente cada cambio por versión.

Estas herramientas cubren directamente las actividades SCM Control, SCM Status Accounting y SCM Auditing de IEEE 828-2012, los procesos 6.3.5/6.3.8/6.4.9 de ISO 12207, las áreas CM/PQA/VV de CMMI, y las cláusulas §8.5.6, §8.6 y §9.1 de ISO 9001:2015.

9.3 ¿La documentación es consistente con los cuatro estándares?

La documentación es parcialmente consistente. Mantenimientos.md actúa como el SCM Plan informal del proyecto: documenta los cuatro tipos de mantenimiento, los CIs afectados, la justificación de cada cambio y la evidencia técnica. El README cubre ejecución, endpoints y seguridad básica. Sin embargo, se identifican brechas comunes a los cuatro marcos normativos:

- Falta .env.example publicado y completo en todos los subsistemas.
- API_DOCS.md es básico en varios CRUDs y no documenta cuerpos de error ni ejemplos de respuesta.
- No existe CHANGELOG.md que registre el historial de versiones y decisiones de arquitectura.
- La deuda técnica de las 4 tablas en epn-event-manager no está registrada formalmente como Change Request / no conformidad pendiente.

Para alcanzar consistencia plena con IEEE 828-2012, ISO/IEC 12207 §6.3.6, CMMI CM/GOV e ISO 9001 §7.5, cada CI de documentación debe actualizarse en cada ciclo y vincularse al Issue o Change Request que originó el cambio.

10. Conclusiones generales

10.1 Sobre la Identificación de la Configuración (SCM Identification)

El ecosistema YO-SOY-CRUD cumple plenamente con el requisito de identificación: los CIs están nombrados de forma única y trazable, las entidades de negocio reciben identificadores generados programáticamente (TRN-XXXX, MSN-XXXX), y el CI de configuración de entorno está separado del código fuente mediante .env.example. El inventario de CIs documentado en este informe satisface el requisito de catálogo explícito exigido por IEEE 828-2012 y el proceso 6.3.5 de ISO/IEC 12207.

10.2 Sobre el Control de Cambios (SCM Control)

Los cambios aplicados siguen una taxonomía formal de cuatro tipos (correctivo, adaptativo, perfectivo, preventivo) documentada en Mantenimientos.md. Sin embargo, el control es parcial porque los cambios no pasan por un flujo formal de Change Request con Issue, rama y PR en el repositorio. Este es el principal gap respecto a IEEE 828-2012, ISO 9001 §8.5.6 y el área CM de CMMI, que exigen que todo cambio sobre un CI sea evaluado, aprobado e implementado de forma controlada y auditable.

10.3 Sobre el Registro de Estado (SCM Status Accounting)

Los logs JSON estructurados con niveles de severidad e ISO 8601 proporcionan una pista de auditoría completa y machine-readable. Las tablas de eventos del hub permiten reconstruir la línea de tiempo completa de cualquier operación. Los endpoints /stats y /health ofrecen reportes de estado en tiempo real. Esta es, de forma consistente en los cuatro informes, el área de mayor cumplimiento del ecosistema, satisfaciendo simultáneamente IEEE 828 SCM Status Accounting, ISO 12207 §6.4.12, CMMI MPM e ISO 9001 §9.1.

10.4 Sobre la Auditoría de Configuración (SCM Auditing)

Las suites de pruebas unitarias y de integración constituyen la auditoría de configuración funcional más directa del ecosistema: verifican que los CIs de validación cumplen su especificación en todos los escenarios límite. La autenticación por API Key, el ValidationPipe global y los health checks complementan la auditoría con controles

de acceso y disponibilidad continuos, satisfaciendo IEEE 828 SCM Auditing, ISO 12207 §6.4.9, CMMI VV/PQA e ISO 9001 §8.6.

10.5 Valoración global

El nivel de cumplimiento con los cuatro estándares analizados puede estimarse como parcial-alto para fines académicos: los controles técnicos de SCM existen y son funcionales en todos los subsistemas revisados, pero requieren formalización documental (CHANGELOG.md, PR templates, .env.example completo), automatización de la auditoría en CI/CD, y registro formal de la deuda técnica arquitectural (las 4 tablas de epn-event-manager) como Change Request / no conformidad pendiente.

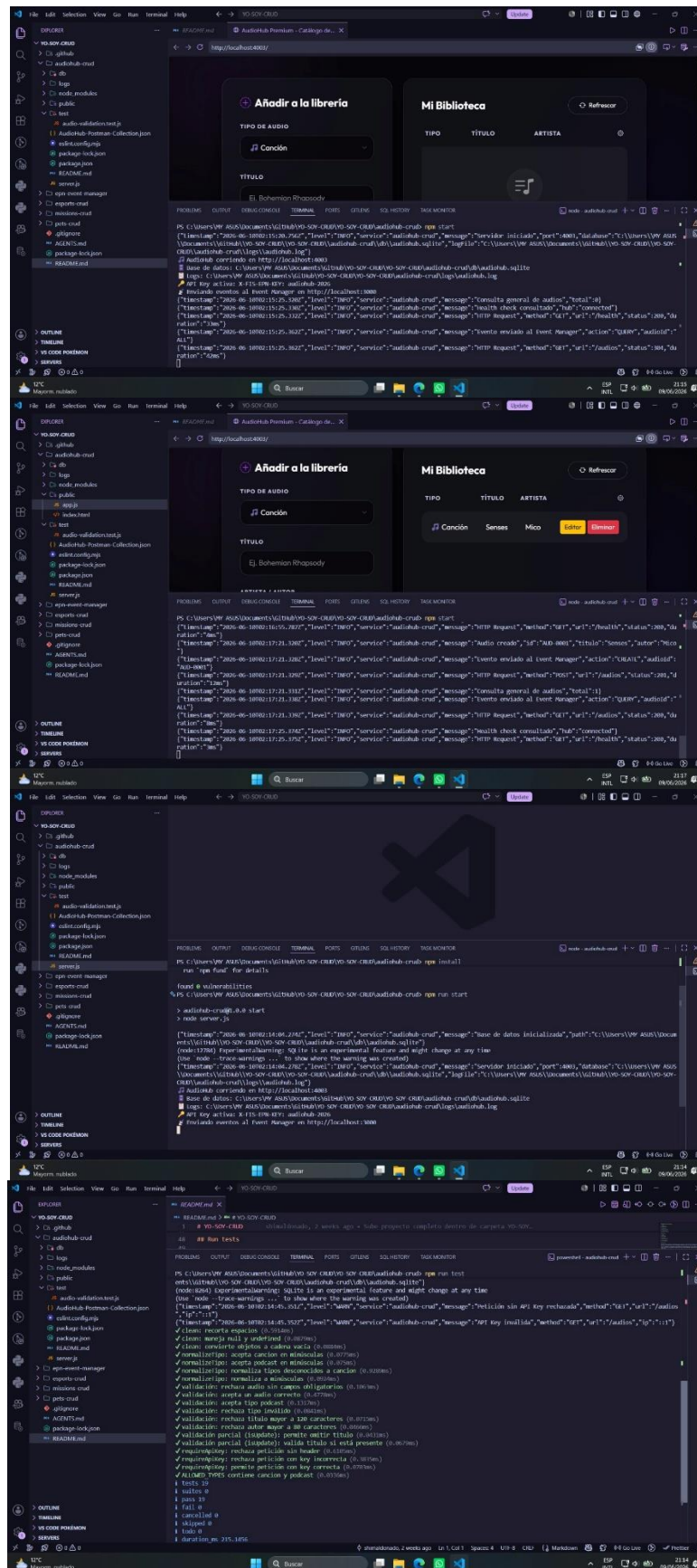
La aplicación conjunta de IEEE 828-2012, ISO/IEC/IEEE 12207:2017, CMMI e ISO 9001:2015 en este ecosistema no es una formalidad documental, sino una práctica integrada en el propio código: cada función de sanitización es simultáneamente un control de integridad de CI (IEEE 828), un proceso de verificación (ISO 12207), una práctica de aseguramiento de calidad (CMMI PQA) y un control de producción (ISO 9001 §8.5.1). Cada tabla de eventos es un registro de estado (Status Accounting, MPM, §9.1), y cada prueba unitaria es una auditoría automatizada de la configuración (SCM Auditing, VV, §8.6). Las tres acciones preventivas propuestas en la sección 8 (control formal de cambios, pipeline CI/CD y documentación viva) cubren de forma transversal las brechas identificadas en los cuatro marcos normativos.

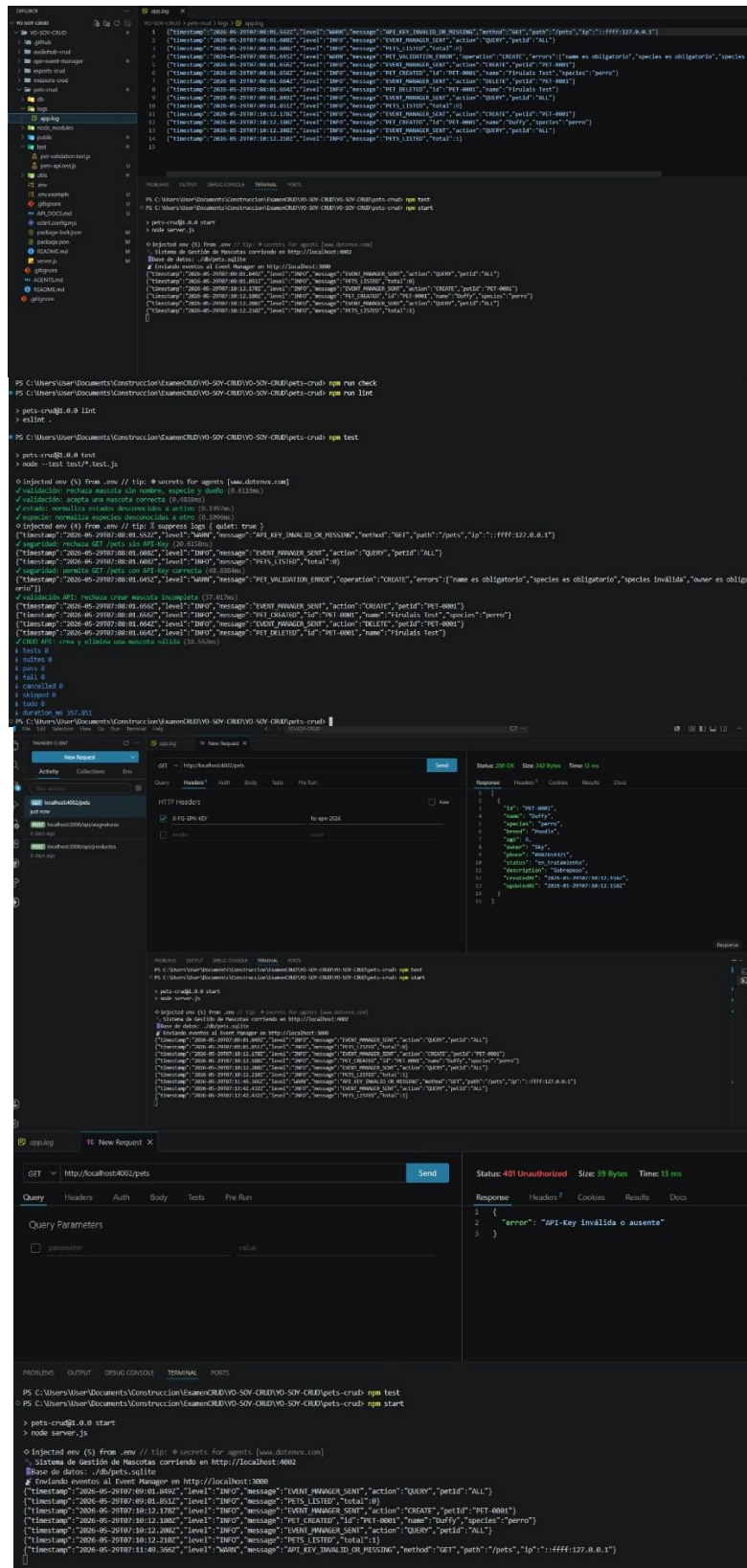
11. Bibliografía

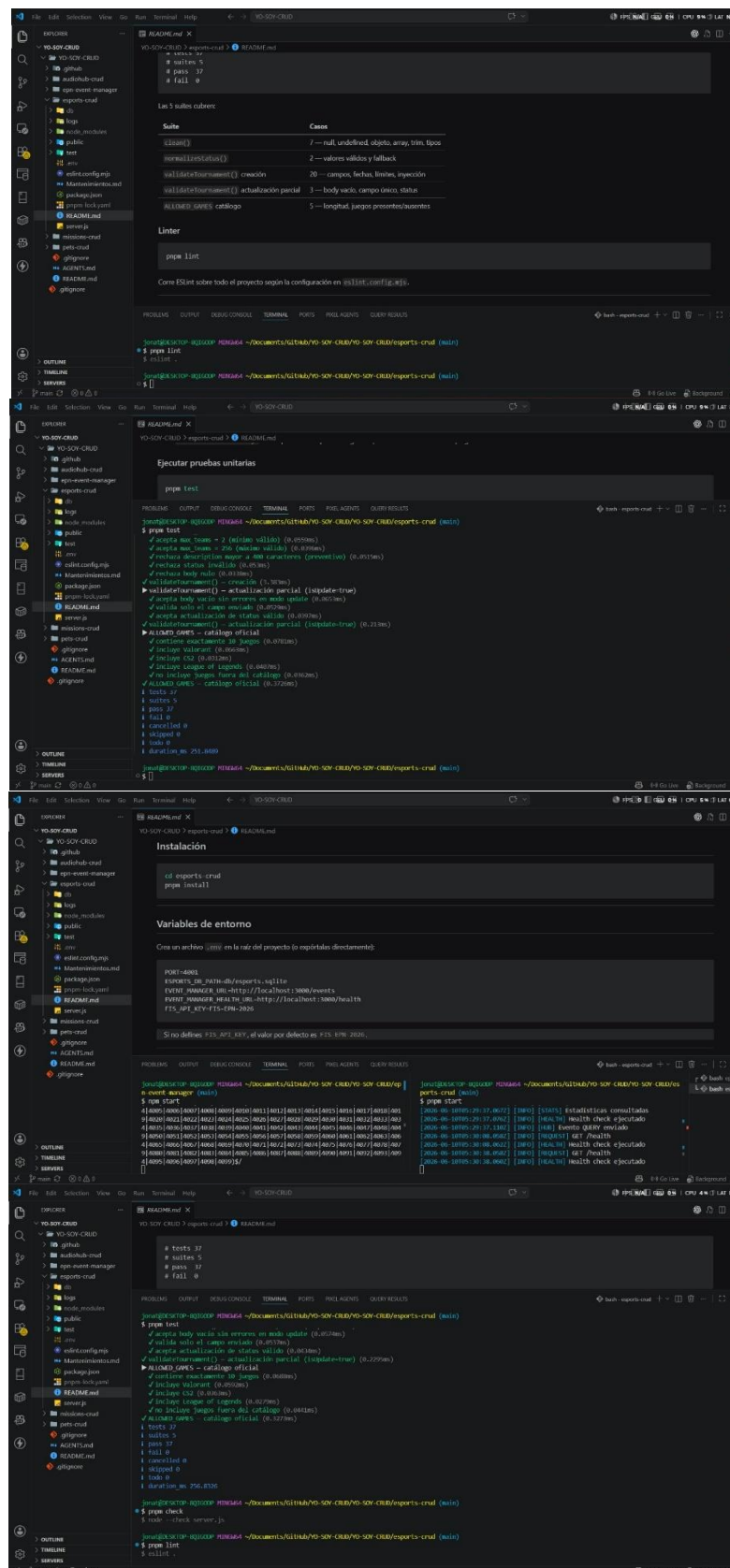
1. IEEE Computer Society, "IEEE 828-2012 — Standard for Configuration Management in Systems and Software Engineering," IEEE, 2012. doi: 10.1109/IEEESTD.2012.6170935.
2. International Organization for Standardization, "ISO/IEC/IEEE 12207:2017 — Systems and software engineering — Software life cycle processes," ISO, 2017. Disponible en: <https://www.iso.org/standard/63712.html>.
3. International Organization for Standardization, "ISO 9001:2015 — Quality management systems — Requirements," ISO, 2015. Disponible en: <https://www.iso.org/standard/62085.html>.
4. CMMI Institute, "CMMI for Development v2.0," CMMI Institute, 2018. Disponible en: <https://cmmiinstitute.com>.
5. ISACA, "CMMI Model Quick Reference Guide," ISACA, 2024. Disponible en: <https://www.isaca.org/resources/reference-guide/cmmi-model-quick-reference-guide>.
6. Pressman, R. S., y Maxim, B. R., Ingeniería del Software: Un Enfoque Práctico, 9.ª ed. McGraw-Hill Education, 2021.
7. SWEBOK Guide v4.0, "Guide to the Software Engineering Body of Knowledge," IEEE Computer Society, 2024.
8. Facultad de Ingeniería de Sistemas — EPN, "Gestión de Configuración y Calidad: Capítulo 2 — Procesos, Estándares y Mantenimiento Aplicado," material de clase, 2026.
9. YO-SOY-CRUD, Repositorio del proyecto: github.com/JonathanGuaman2004/YO-SOY-CRUD, GitHub, 2026 (incluye esports-crud, audiodhub-crud, missions-crud, pets-crud, epn-event-manager y sus respectivos Mantenimientos.md, suites de pruebas y documentación).

12. Anexos

Audiodhub-crud

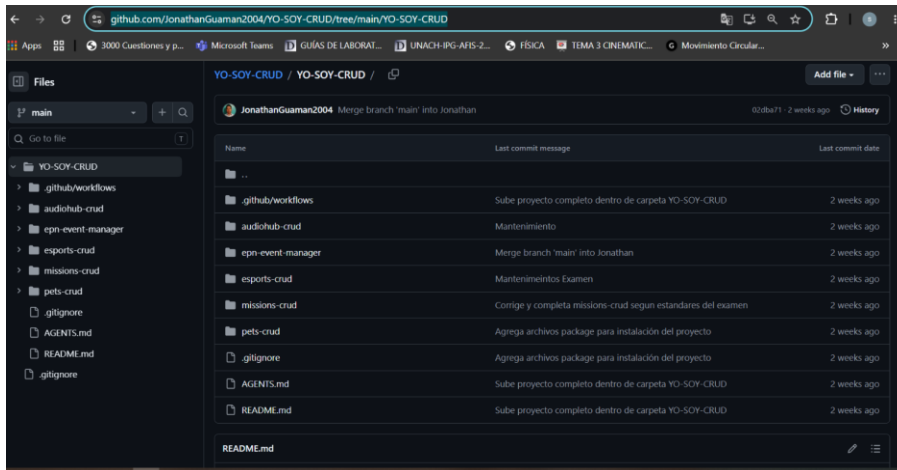






The image shows a complex web application interface for "SPACE MISSION CONTROL". At the top, there's a navigation bar with tabs for "package.json", "missions.sql", "npm scripts", and "localhost:3000/events". Below this is a sidebar with a file explorer showing a project structure with folders like "YO-SOY-CRUD", "db", "docs", "logs", "node_modules", "postman", "public", "test", "env", "eslint.config.mjs", "package.json", "package-lock.json", "README.md", "server.js", "pets-crud", "gignore", "AGENTS.md", "AGENTS.md", "gignore", "OUTLINE", and "TIMELINE". The main content area is divided into several sections. The top section is a table of missions with columns: id, name, agency, type, vehicle, and date. It lists three missions: MSN-0001 (sorhti), MSN-0002 (spaceNitra), and MSN-0003 (salomon). Below the table is a "PROBLEMS" section with a "OUTPUT" tab showing a list of errors and warnings. The bottom section is a "TIMELINE" showing a list of events with columns for time, event, and user. The interface is dark-themed and includes various UI elements like buttons, dropdowns, and a search bar.

Events



Audit log

